Algoritmos de Ordenação (insertion, selection, bubble)

Estruturas de Dados

adalberto@computacao.ufcg.edu.br

Ordenação

$<a_1,a_2,a_3...a_n>$ ⇨ ordenação ⇨

Fundamental para organizar a informação

- Bancos de dados
- Sistemas Operacionais
- Engenhos de Busca
- Redes Sociais
- Compiladores

Propriedades importantes

- Algoritmos estável (preserva a ordem de elementos iguais)
- Algoritmo in-place (não usa memória extra)

Algoritmos de ordenação

- Por comparação
  - Simples
  - Dividir para conquistar
- Em tempo linear
  - Preço a pagar (memória extra)

Bubble sort

- Baseado na troca de elementos adjacentes
- Cada iteração leva o elemento maior para o final do vetor

Bubble sort

```
bubbleSort( A : list of sortable items )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in 1 to n-i{
      if(A[i] > A[i+1]){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

```
swap(A[i],A[j]){
  int temp = A[i];
  A[i] = A[j];
  A[j] = temp;
}
```

Bubble sort

```
bubbleSort( A : list of sortable items )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in 1 to n-i{
      if(A[i] > A[i+1]){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

Pior caso

i=1 → **n-1**
i=2 → **n-2**
i=3 → **n-3**
...
i=n-1 → **1**

Bubble sort

```
bubbleSort( A : list of sortable items )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in 1 to n-i{
      if(A[i] > A[i+1]){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

Pior caso

i=1 → **n-1**
i=2 → **n-2**
i=3 → **n-3**
...
i=n-1 → **1**

n + n +... + n - (1+2+3+...)

$$n^2 - S_n$$

$$n^2 - n(n+1)/2$$

$$(n^2 - 1)/2 = \Theta(n^2)$$

Bubble sort

```
bubbleSort( A : list of sortable items )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in 1 to n-i{
      if(A[i] > A[i+1]){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

Melhor caso?

Selection sort

- Baseado na seleção do maior e seu posicionamento no final
- Primeiro seleciona, depois leva para a posição correta

Selection sort

```
SELECTION-SORT(A)
   n = length(A)
   for i ← 1 to n  {
        min ← i
        for j ← (i + 1) to n  {
            if A[j] < A[min] {
                min ← j
            }
        }
        swap (A[i], A[min])
  }
}
```

Selection sort

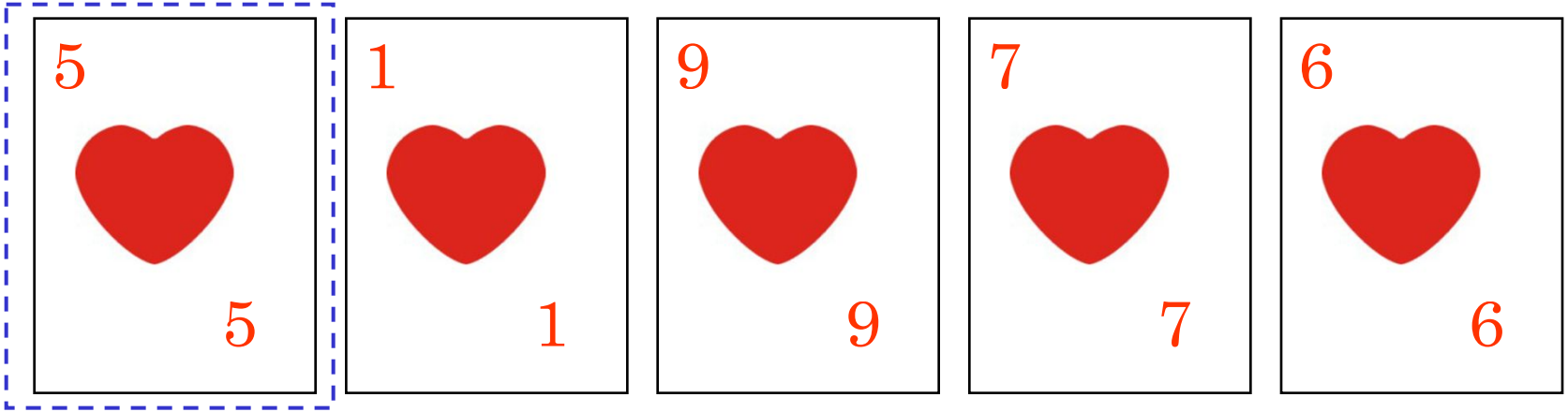
```
SELECTION-SORT(A)
   n = length(A)
   for i ← 1 to n  {
        min ← i
        for j ← (i + 1) to n {
            if A[j] < A[min] {
                min ← j
            }
        }
        swap (A[i], A[min])
  }
}
```

(n – 1)
(n – 2)
(n – 3)
…
(n – n)

$$n^2 - S_n$$

$$(n^2 - 1)/2 = \Theta(n^2)$$

Insertion sort



- Baseado na ideia de **inserir** um elemento em uma lista ordenada

Insertion sort

```
INSERTION-SORT(A, n)
    for j ← 2 to n do
        key ← A[j]
        i ← j-1
        while i > 0 and A[i] > key do
              A[i+1] ← A[i]
              i ← i-1
        A[i+1] = key
```

Insertion sort

```
INSERTION-SORT(A, n)
    for j ← 2 to n do
        key ← A[j]
        i ← j-1
        while i > 0 and A[i] > key do
              A[i+1] ← A[i]
              i ← i-1
        A[i+1] = key
```

(1)
(2)
(3)
…
(n)

Comparação

Algoritmo	melhor tempo	tempo médio	pior tempo	espaço
Insertion sort	O(n)	O(nˆ2)	O(nˆ2)	in-place
Selection sort	O(nˆ2)	O(nˆ2)	O(nˆ2)	in-place
Bubble sort	O(n)	O(nˆ2)	O(nˆ2)	in-place

Questões de implementação

Como adaptar algoritmos para ordenar tipos genéricos?

Como ordenar apenas uma faixa de um array?

Questões de implementação

```
bubbleSort( A : list of sortable items )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in 1 to n-i{
      if(A[i] > A[i+1]){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

Como adaptar algoritmos para ordenar tipos genéricos?

Questões de implementação

```
bubbleSort( A : list of Comparable items )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in 1 to n-i{
      if(A[i].compareTo(A[i+1])) > 0){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

Como adaptar algoritmos para ordenar tipos genéricos?

Questões de implementação

```
bubbleSort( A : list of Comparable items )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in 1 to n-i{
      if(A[i].compareTo(A[i+1])) > 0){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

Como ordenar apenas uma faixa de um array?

Questões de implementação

```
bubbleSort( A : list of Comparable items
            leftIndex, rightIndex )
  swapped = true;
  n = length(A)
  while swapped {
    swapped = false;
    for each i in leftIndex to rightIndex-i{
      if(A[i].compareTo(A[i+1])) > 0){
        swap(A[i],A[i+1])
        swapped = true
      }
    }
  }
}
```

Como ordenar apenas uma faixa de um array?